

HOMEWORK 4 - Image Sharpening

Task 1 - Spatial Domain

- Content: Laplacian Filter in Spatial Domain
- Implement Convolution by yourself:
 - **Do NOT** use built-in convolution functions (e.g., `cv2.filter2D`, `scipy.signal.convolve2d`).
 - You must handle the image boundaries (e.g., Zero-padding, Replicate-padding).
- Implement Laplacian Filter by yourself:
 - You can experiment with different kernel sizes and types (e.g., 4-neighbor or 8-neighbor kernels).
 - Combine the Laplacian edges with the original image to output the sharpened result.

Task 2 - Frequency Domain

- Content: Laplacian Filter in Frequency Domain
- Step 1: Transform to Frequency Domain
 - Get the spectrum using Fourier Transform (e.g., `np.fft.fft2`)
 - Hint: Remember to shift the zero-frequency component to the center
- Step 2: Apply Filter on Spectrum
 - Construct the Laplacian filter $H(u, v)$ in the frequency domain
 - Multiply the filter with the image spectrum
- Step 3: Inverse Transform
 - Shift the zero-frequency component back
 - Convert the new spectrum back to the spatial domain using Inverse Fourier Transform
 - Take the real part of the result

TA's Hints

Common Pitfalls:

- **Data Types Matter:** Images read by OpenCV/PIL are usually `uint8`. If you apply a Laplacian filter directly, negative values will cause overflow/underflow artifacts. Convert your image to `float32` or `float64` before processing.
- **Normalization:** After filtering, your values might exceed the `[0, 255]` range. Remember to `np.clip` or normalize your final result before saving it back to `uint8`.
- **Spectrum Visualization:** If you want to visualize the Fourier spectrum in your report, apply a logarithmic transformation (e.g., `np.log(1 + np.abs(spectrum))`), otherwise it will just look like a black image with a tiny white dot.

Implementation & Grading Policy

- **Implement Policy**

- Find images by yourself: Choose at least 2-3 images with clear textures/edges to demonstrate the sharpening effect clearly.
- Coding Restrictions: Only basic matrix operations are allowed for the core logic. (FFT libraries like `numpy.fft` are allowed for Step 2).

- **Grading Policy (Total: 100 pts)**

- Spatial Domain Implementation: 45 pts
- Frequency Domain Implementation: 45 pts
- Report: 10 pts
- Format Penalty: -10 pts (Wrong file naming, missing files, or unzippable formats).

Submission Guidelines

- **File Structure**

- |—— 109xxxxxx.pdf (Report)
- L—— 109xxxxxx.zip (Source Code)
- Please compress your files into a .zip format (Do NOT use .rar or .7z).

- **Report Requirements**

- Method: Explain your convolution padding strategy and how you constructed the frequency filter.
- Result: Display original images vs. sharpened images (both domains). Provide a clear visual comparison.
- Discussion: Discuss any differences between the two domains (e.g., computation time, ringing artifacts) and challenges faced.

- **Deadline: 5/12 Tue. 23:59**